Exception & System Call

開發學生:黃彥筑、許展富、王冠人

高效能計算實驗室

國立中正大學 資訊工程學系

Exceptions

- **Exceptions are usually used to handle unexpected events which arise during the execution of a program**
- An exception is an **unexpected or special event** that causes the CPU to stop normal execution and transfer control to the kernel to handle it.

Exception Types

Faults

- Exception BEFORE instruction completes.
- Example:
  - Division by Zero → arithmetic error
- CPU saves PC pointing to the faulting instruction. After handling, retry same instruction.

Traps

- Exception AFTER instruction completes.
- Example:
  - Syscall → SWI/SVC instruction
- CPU saves PC pointing to NEXT instruction. After handling, continue with next instruction.

Exception Types

Aborts

- Severe error, cannot recover process or system must terminate.
- Example:
  - Bus error
- Usually kills the process (SIGABRT / SIGBUS).

Exception in Linux (ARM32)

- When an exception is generated, a function: vector_swi() is called.
- vector_swi() is defined in <kernel source code/arch/arm/kernel/entry-common.S>.
- vector_swi() gets the system call number in the R7 register and finds the system call address in the sys_call_table and invokes it.
- Registers R0 ~ R6 are used to send arguments to the system calls.

(ARM32)

User Space | Kernel Space

SWI instruction → ARM Exception Vector

↓

vector_swi()

(entry-common.S, arch/arm/kernel)

↓

Read syscall number (R7)

↓

sys_call_table lookup

↓

Execute sys_xxx() handler

↓

Return result in R0

vector_swi() is defined in <kernel source code/arch/arm/kernel/entry-common.S>

https://github.com/torvalds/linux/blob/master/arch/arm/kernel/entry-common.S

master linux / arch / arm / kernel / entry-common.S

Code Blame

```
170             .align        5
171     ENTRY(vector_swi)
172     #ifdef CONFIG_CPU_V7M
173             v7m_exception_entry
174     #else
175             sub         sp, sp, #PT_REGS_SIZE
176             stmia   sp, {r0 - r12}                      @ Calling r0 - r12
177     3:
178      ARM(   add         r8, sp, #S_PC          )
179      ARM(   stmdb   r8, {sp, lr}^          )       @ Calling sp, lr
180      THUMB( mov         r8, sp                  )
181      THUMB( store_user_sp_lr r8, r10, S_SP    )       @ calling sp, lr
182             mrs     saved_psr, spsr                  @ called from non-FIQ mode, so ok.
183      TRACE( mov         saved_pc, lr            )
184             str         saved_pc, [sp, #S_PC]               @ Save calling PC
185             str         saved_psr, [sp, #S_PSR]               @ Save CPSR
186             str         r0, [sp, #S_OLD_R0]               @ Save OLD_R0
187     #endif
188             reload_current r10, ip
189             zero_fp
```

(ARM64)

User Space | Kernel Space

SVC #0 instruction → Exception Vector Table (el1)

↓

el0_sync_compat / el0_svc
(arch/arm64/kernel/entry.S)

↓

el0_svc_common()
(arch/arm64/kernel/syscall.c)

↓

Read syscall number (X8)

↓

sys_call_table lookup

↓

Execute sys_xxx() handler

↓

Return result in X0

Outline

- Environment
- Goal
- Steps
  - Initialization
  - Observation a system call
  - Write a system call
- Reference

Environment

- PC x 1
  - Linux Ubuntu
    - Editor, arm cross compiler
- Raspberry Pi

Goal

- Observe the implementation of system calls.
- Add a new system call.

Step1 - Initialization

- System boot
  - Start kernel
- Prepare
  - Download kernel
    - 使用buildroot所提供的source code
    - buildroot/output/build/linux-custom
  - Cross toolchain
    - 使用buildroot所編譯好的cross toolchain或是自己所編譯的cross toolchain
    - toolchain 執行檔所在目錄 buildroot/output/host/bin/

假設buildroot的configure file為
- raspberrypi3_defconfig (32-bit Linux)
- raspberrypi3_64_defconfig (64-bit Linux)

Step2 - Observation of a system call

- In general, a user level process is not supposed to access kernel.
- Linux uses software interrupt to generate exception to implement system calls.
- The user-level calling process fills the registers with the appropriate values and then calls a special instruction which jumps to a previously defined location in the kernel.

```
svc 0x0
```

Step2 - Observation of a system call

- Use a program calling getpid() to investigate its assembly
  - Returns the Process ID: a unique integer the kernel assigns to every running process.

```
#include <sys/types.h>
#include <unistd.h>

int main(void)
{
    getpid();
    return 0;
}
```

Step2 - Observation of a system call

- Cross-compile with static link

Arm32: arm
Arm64: aarch64

```
$./buildroot/output/host/bin/arm-linux-gcc -g -static -o test.exe test.c
```

- Disassemble the executable code: test.exe

```
$./buildroot/output/host/bin/arm-linux-objdump -d test.exe > assembly
```

Step2 - Observation of a system call

- Find an svc(swi) instruction for the system call

output : vi — Konsole

File Edit View Bookmarks Settings Help

```
00010220 <frame_dummy>:
   10220:       e59f301c        ldr     r3, [pc, #28]   ; 10244 <frame_dummy+0x24>
   10224:       e3530000        cmp     r3, #0
   10228:       0a000004        beq     10240 <frame_dummy+0x20>
   1022c:       e92d4010        push    {r4, lr}
   10230:       e59f1010        ldr     r1, [pc, #16]   ; 10248 <frame_dummy+0x28>
   10234:       e59f0010        ldr     r0, [pc, #16]   ; 1024c <frame_dummy+0x2c>
   10238:       e320f000        nop     {0}
   1023c:       e8bd4010        pop     {r4, lr}
   10240:       eaffffd7        b       101a4 <register_tm_clones>
   10244:       00000000        .word   0x00000000
   10248:       0002c14c        .word   0x0002c14c
   1024c:       0001ba00        .word   0x0001ba00

00010250 <main>:
   10250:       e92d4800        push    {fp, lr}
   10254:       e28db004        add     fp, sp, #4
   10258:       eb00019f        bl      108dc <__GI_getpid>
   1025c:       e3a03000        mov     r3, #0
   10260:       e1a00003        mov     r0, r3
   10264:       e8bd8800        pop     {fp, pc}

00010268 <__GI_abort>:
   10268:       e52de004        push    {lr}            ; (str lr, [sp, #-4]!)
   1026c:       e59f00f0        ldr     r0, [pc, #240]  ; 10364 <__GI_abort+0xfc>
   10270:       e24dd024        sub     sp, sp, #36     ; 0x24
   10274:       e08f0000        add     r0, pc, r0
   10278:       eb00004d        bl      103b4 <__pthread_mutex_lock>
   1027c:       e3a02000        mov     r2, #0
   10280:       e28d1004        add     r1, sp, #4
```

output : vi — Konsole

File Edit View Bookmarks Settings Help

Arm32 為 20

```
   108a4:       0001bb80        .word   0x0001bb80
   108a8:       0001bb14        .word   0x0001bb14
   108ac:       0001b9bc        .word   0x0001b9bc
   108b0:       000000e4        .word   0x000000e4
   108b4:       00000070        .word   0x00000070
   108b8:       0001b80c        .word   0x0001b80c
   108bc:       0001b808        .word   0x0001b808
   108c0:       0001ba8c        .word   0x0001ba8c
   108c4:       0001b7d8        .word   0x0001b7d8
   108c8:       0001b7d8        .word   0x0001b7d8
   108cc:       000000d0        .word   0x000000d0
   108d0:       000000a0        .word   0x000000a0
   108d4:       00000090        .word   0x00000090
   108d8:       0001b6a8        .word   0x0001b6a8

000108dc <__GI_getpid>:
   108dc:       e92d4080        push    {r7, lr}
   108e0:       e3a07014        mov     r7, #20
   108e4:       ef000000        svc     0x00000000
   108e8:       e8bd8080        pop     {r7, pc}

000108ec <__GI_raise>:
   108ec:       e92d408f        push    {r0, r1, r2, r3, r7, lr}
   108f0:       e1a0c000        mov     ip, r0
   108f4:       e3e01001        mvn     r1, #1
   108f8:       e1a0e00d        mov     lr, sp
   108fc:       e3e00102        mvn     r0, #-2147483648        ; 0x80000000
   10900:       e1a0200e        mov     r2, lr
   10904:       e1cd00f8        strd    r0, [sp, #8]
   10908:       e3a03008        mov     r3, #8
   1090c:       e3a00000        mov     r0, #0
```

pschen@debian12-64: ~/WORK/embedded_exp/syscall_exp

```
  40a418:       d4207d00        brk     #0x3e8
  40a41c:       17fffff8        b       40a3fc <_exit+0x2c>
  40a420:       d503201f        nop
  40a424:       d503201f        nop
  40a428:       d503201f        nop
  40a42c:       d503201f        nop
  40a430:       d503201f        nop
  40a434:       d503201f        nop
  40a438:       d503201f        nop
  40a43c:       d503201f        nop

000000000040a440 <__getpid>:
  40a440:       d503201f        nop
  40a444:       d2801588        mov     x8, #0xac                       // #172
  40a448:       d4000001        svc     #0x0
  40a44c:       d65f03c0        ret
  40a450:       d503201f        nop
  40a454:       d503201f        nop
  40a458:       d503201f        nop
  40a45c:       d503201f        nop
  40a460:       d503201f        nop
  40a464:       d503201f        nop
  40a468:       d503201f        nop
/getpid
```

Arm64 為 0xac (172)

```
$ cd linux-custom/arch/arm/tools
$ vi syscall_32.tbl
```

Arm32

tools : vi — Konsole

File Edit View Bookmarks Settings Help

```
#
# Linux system call numbers and entry vectors
#
# The format is:
# <num> <abi>   <name>                [<entry point>                [<oabi compat entry point>]]
#
# Where abi is:
#  common - for system calls shared between oabi and eabi (may have compat)
#  oabi   - for oabi-only system calls (may have compat)
#  eabi   - for eabi-only system calls
#
# For each syscall number, "common" is mutually exclusive with oabi and eabi
#
0       common  restart_syscall         sys_restart_syscall
1       common  exit                    sys_exit
2       common  fork                    sys_fork
3       common  read                    sys_read
4       common  write                   sys_write
5       common  open                    sys_open
6       common  close                   sys_close
# 7 was sys_waitpid
8       common  creat                   sys_creat
9       common  link                    sys_link
10      common  unlink                  sys_unlink
11      common  execve                  sys_execve
12      common  chdir                   sys_chdir
13      oabi    time                    sys_time32
14      common  mknod                   sys_mknod
15      common  chmod                   sys_chmod
16      common  lchown                  sys_lchown16
# 17 was sys_break
```

```
$ cd linux-custom/arch/arm/tools
$ vi syscall_32.tbl
```

Arm32

tools : vi — Konsole

File Edit View Bookmarks Settings Help

```
#
# For each syscall number, "common" is mutually exclusive with oabi and eabi
#
0       common  restart_syscall         sys_restart_syscall
1       common  exit                    sys_exit
2       common  fork                    sys_fork
3       common  read                    sys_read
4       common  write                   sys_write
5       common  open                    sys_open
6       common  close                   sys_close
# 7 was sys_waitpid
8       common  creat                   sys_creat
9       common  link                    sys_link
10      common  unlink                  sys_unlink
11      common  execve                  sys_execve
12      common  chdir                   sys_chdir
13      oabi    time                    sys_time32
14      common  mknod                   sys_mknod
15      common  chmod                   sys_chmod
16      common  lchown                  sys_lchown16
# 17 was sys_break
# 18 was sys_stat
19      common  lseek                   sys_lseek
20      common  getpid                  sys_getpid
21      common  mount                   sys_mount
22      oabi    umount                  sys_oldumount
23      common  setuid                  sys_setuid16
24      common  getuid                  sys_getuid16
25      oabi    stime                   sys_stime32
26      common  ptrace                  sys_ptrace
27      oabi    alarm                   sys_alarm
```

```
$ cd linux-custom/arch/arm64/tools
$ vi syscall_64.tbl
```

Arm64

Code | Blame

```
203     165 common  getrusage           sys_getrusage           compat_sys_getrusage
204     166 common  umask               sys_umask
205     167 common  prctl               sys_prctl
206     168 common  getcpu              sys_getcpu
207     169 time32  gettimeofday            sys_gettimeofday        compat_sys_gettimeofday
208     169 64  gettimeofday            sys_gettimeofday
209     170 time32  settimeofday            sys_settimeofday        compat_sys_settimeofday
210     170 64  settimeofday            sys_settimeofday
211     171 time32  adjtimex            sys_adjtimex_time32
212     171 64  adjtimex            sys_adjtimex
213     172 common  getpid              sys_getpid
214     173 common  getppid             sys_getppid
215     174 common  getuid              sys_getuid
216     175 common  geteuid             sys_geteuid
217     176 common  getgid              sys_getgid
218     177 common  getegid             sys_getegid
219     178 common  gettid              sys_gettid
```

Step3 - Write a system call

- ARM64

Steps to Add a Syscall in ARM64

- Register syscall => arch/arm64/tools/syscall_64.tbl
- Declare syscall => include/linux/syscalls.h
- Implement syscall handler => kernel/sys.c
- Build and test => user-space test program

Register System Call (ARM64)

1. Add your system call to the kernel's system call table.

```
$vi arch/arm64/tools/syscall_64.tbl
```

```
454     common  futex_wake                      sys_futex_wake
455     common  futex_wait                      sys_futex_wait
456     common  futex_requeue                   sys_futex_requeue
457     common  statmount                       sys_statmount
458     common  listmount                       sys_listmount
459     common  lsm_get_self_attr               sys_lsm_get_self_attr
460     common  lsm_set_self_attr               sys_lsm_set_self_attr
461     common  lsm_list_modules                sys_lsm_list_modules
462     common  mseal                           sys_mseal
463     common  mysys                           sys_mysys
~
"syscall_64.tbl" 406 lines, 17295 bytes written
```

Declare System Call (ARM64)

2. Add a corresponding function prototype for your system call to the header file of system calls.

```
$vi include/linux/syscalls.h
```

```
/*
 * These syscall function prototypes are kept in the same order as
 * include/uapi/asm-generic/unistd.h. …
 * …
 */
#ifndef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
…
asmlinkage long sys_ni_syscall(void);

asmlinkage long sys_mysys(void);
#endif /* CONFIG_ARCH_HAS_SYSCALL_WRAPPER */
```

Implement System Call Handler (ARM64)

3. Change your working directory to the Linux source's root directory

```
$cd buildroot/output/build/linux-custom
```

Implement System Call Handler (ARM64)

4. Add your system call's code in sys.c

```
$vi kernel/sys.c
```

Or add an additional file => kernel/my_syscall.c (需適當修改Makefile)

```
/* sys_mysys - example syscall
 * No arguments.
 */

SYSCALL_DEFINE0(mysys)
{
    printk("Hello, I am System Call!\n");
    return 0;
}
```

ReCompile Linux Kernel

5. Compile Linux kernel.

- Change to buildroot's root directory
- Compile linux kernel image

```
$make linux-rebuild
```

- 產生新的kernel image "Image"
- 將Image複製到SDcard的FAT partition (BOOT)

Test System Call

6. Write a program to use the system call
   - test.c

test.c

```
#include <sys/syscall.h>
#include <stdio.h>
#include <unistd.h>

#define __NR_mysys 463

int main(void)
{
    long ret;

    ret = syscall(__NR_mysys);
    printf("return value %d\n", ret); // expect 0.

    return 0;
}
```

```
/* glibc ARM64 syscall() implementation */
ENTRY (syscall)
   mov    x8, x0              /* syscall number → X8 */
   mov    x0, x1              /* arg1 → X0 */
   mov    x1, x2              /* arg2 → X1 */
   mov    x2, x3              /* arg3 → X2 */
   mov    x3, x4              /* arg4 → X3 */
   mov    x4, x5              /* arg5 → X4 */
   mov    x5, x6              /* arg6 → X5 */
   svc    0x0                 /* trap to kernel */
   cmn    x0, #4095           /* check for error */
   b.hi   SYSCALL_ERROR_LABEL    /* handle error */
   ret                        /* return */
END (syscall)
```

Evaluation (ARM64)

7. Compilation and verification

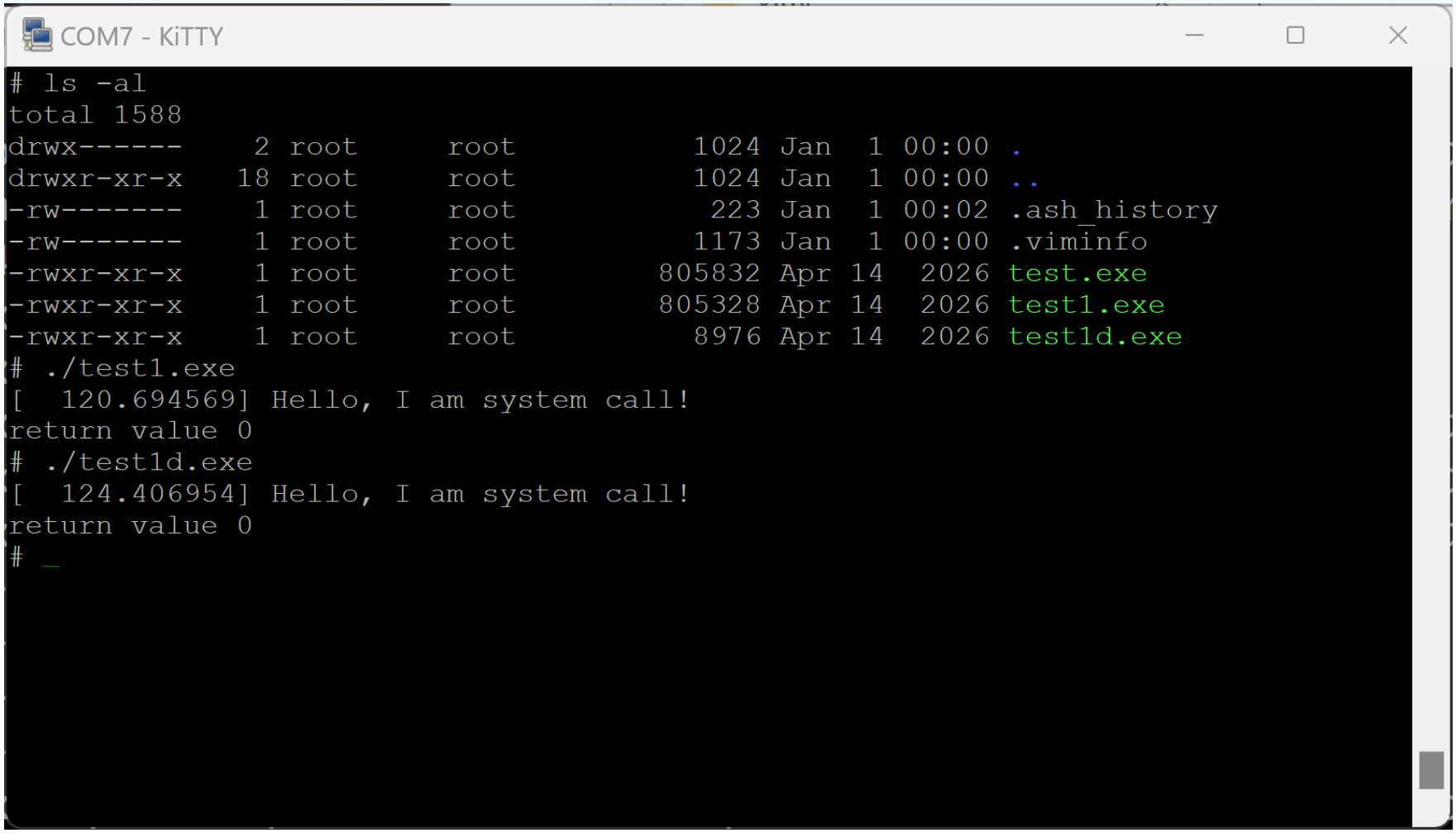
Arm64: aarch64

- Compile test.c for the target, ARM-Linux

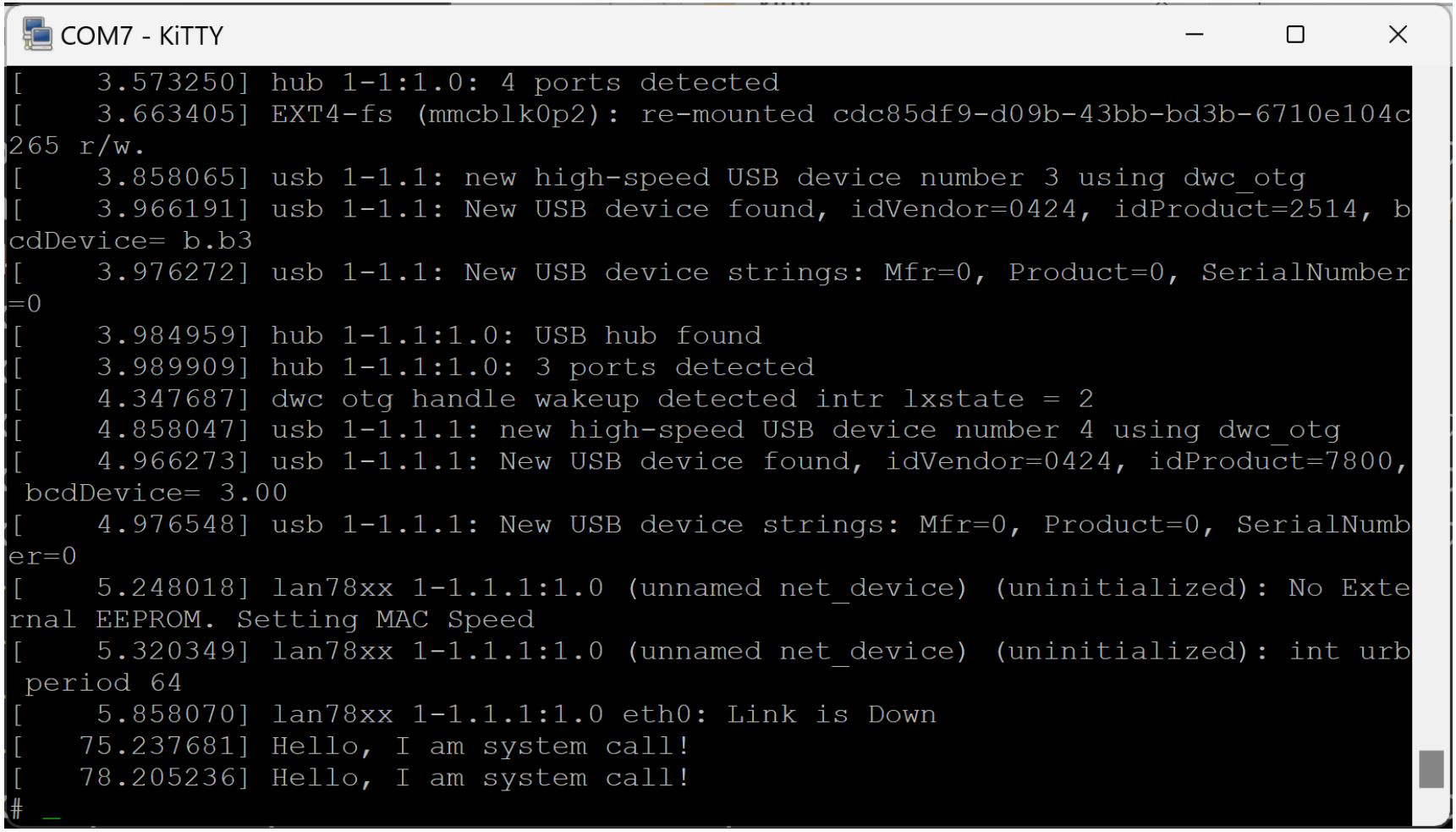
```
$buildroot/output/host/bin/aarch64-linux-gcc -
static -g test.c -o test.exe
```

Execute the user program (ex: test.exe) in Raspberry Pi
(將test.exe複製到Sdcard的 root filesystem partition)

- 重新開機
- 執行 test.exe
- type dmesg in terminal and the system call message will be displayed.


COM7 - KiTTY
ls -al
total 1588
drwx------ 2 root root 1024 Jan 1 00:00 .
drwxr-xr-x 18 root root 1024 Jan 1 00:00 ..
-rw------- 1 root root 223 Jan 1 00:02 .ash_history
-rw------- 1 root root 1173 Jan 1 00:00 .viminfo
-rwxr-xr-x 1 root root 805832 Apr 14 2026 test.exe
-rwxr-xr-x 1 root root 805328 Apr 14 2026 test1.exe
-rwxr-xr-x 1 root root 8976 Apr 14 2026 test1d.exe
./test1.exe
[120.694569] Hello, I am system call!
return value 0
./test1d.exe
[124.406954] Hello, I am system call!
return value 0
#

$ dmesg


COM7 - KiTTY
[3.573250] hub 1-1:1.0: 4 ports detected
[3.663405] EXT4-fs (mmcblk0p2): re-mounted cdc85df9-d09b-43bb-bd3b-6710e104c265 r/w.
[3.858065] usb 1-1.1: new high-speed USB device number 3 using dwc_otg
[3.966191] usb 1-1.1: New USB device found, idVendor=0424, idProduct=2514, bcdDevice= b.b3
[3.976272] usb 1-1.1: New USB device strings: Mfr=0, Product=0, SerialNumber=0
[3.984959] hub 1-1.1:1.0: USB hub found
[3.989909] hub 1-1.1:1.0: 3 ports detected
[4.347687] dwc otg handle wakeup detected intr lxstate = 2
[4.858047] usb 1-1.1.1: new high-speed USB device number 4 using dwc_otg
[4.966273] usb 1-1.1.1: New USB device found, idVendor=0424, idProduct=7800, bcdDevice= 3.00
[4.976548] usb 1-1.1.1: New USB device strings: Mfr=0, Product=0, SerialNumber=0
[5.248018] lan78xx 1-1.1.1:1.0 (unnamed net_device) (uninitialized): No External EEPROM. Setting MAC Speed
[5.320349] lan78xx 1-1.1.1:1.0 (unnamed net_device) (uninitialized): int urb period 64
[5.858070] lan78xx 1-1.1.1:1.0 eth0: Link is Down
[75.237681] Hello, I am system call!
[78.205236] Hello, I am system call!
_

- Q1: 請問sys.c中實作mysys，SYSCALL_DEFINE0()的作用是什麼？

```
/* sys_mysys - example syscall
 * No arguments.
 */

SYSCALL_DEFINE0(mysys)
{
    printk("Hello, I am System Call!\n");
    return 0;
}
```

- Q2: asmlinkage 的描述有什麼作用？

請在實驗報告裡回答。